The Lab Recipe

Sep 27, 2026 · @greg

A paper is a chain of files, each written by a script that recorded what it read, and a number enters the manuscript only through that chain. This page is the recipe for running that chain in the lab; the full version with every command is `RECIPE.md` in Student_Projects, and the projects with their leads are `BOARD.md`.

Roles and the board

Every project on the board names a lead, a co-lead and its members. A project without a co-lead is unstaffed and takes no members.

Role	Owes the project	How many
Lead	the question, the frozen pre-registration, the manuscript; answers project email within two working days; a 15-minute weekly check-in; merges pull requests	one
Co-lead	everything the lead can do; takes over when the lead leaves; second reader on every freeze	one
Members	tasks: annotate, measure, render, check, draft a section; each task is a board line with a name on it	any number

Above the projects: Ashley Schehr (senior scientist, co-PI) screens, Dr Tech (senior clinical PI) confirms clinical reads, and Greg Schwing signs off on every freeze and submission.

Join. Read `BOARD.md`, pick a project, email its lead with the subject `Join: <project id>` and your hours per week. The lead adds you to the card by pull request within the week and hands you one open task.

Leave. A handover pull request (section on leaving below). A member silent for three weeks is re-staffed by the lead.

The shape of a project

A task is the unit of work: one script, one job script, one results file, and the manifest keys that quote it. A task that does not end in a results file is not finished.

```
<project>/
  PREREG.md               what will be measured and what counts as failure;
frozen before the first number
  PREREG.md.sha256        its fingerprint; every producing job checks it and
refuses on mismatch
  DEVIATIONS.md           every departure from PREREG.md, dated, with the
reason
  JOURNAL.md              dated entries: what ran, job ids, what came out,
what is next
  code/                   one script per task; each writes exactly one
results file
  slurm/                  one job script per task; its header says what it
produces and how to resume it
  results/                one JSON per task, each carrying its own _run block
  numbers_manifest.json   key -> results file, JSON pointer, format, job id,
PREREG item
  paper/                  the tex; numbers only as \RESULT{key};
generated/numbers.tex written at build
  RESUME.md               newest-first checkpoint blocks
  env/<name>.lock.txt     the exact environment the results came from (make
env-lock)
```

The board card lives in `board.json` in Student_Projects; `tools/render_board.py` renders `BOARD.md` and refuses a card without a lead, a contact or a known state.

Pre-register and freeze

Write `PREREG.md` before any script produces a number, freeze it with its hash, and never edit it again. Departures go in `DEVIATIONS.md`; a second version is a new file with its own freeze.

Pre-stated rules make a negative result reportable. The render-ablation rule for the SchwingNet paper was written before the numbers; when one corpus came out the other way, that cell went into the manuscript. In the U251 work the primary endpoint's bar was 0.50 in advance; it failed, and the manifest says so.

`PREREG.md` contains:

1. The question, in one sentence a clinician would recognise.
2. Population and inputs: dataset release by version and DOI, files and their hashes, exclusions with reasons.
3. Each measurement as a proposal states it: identifiers, axis, unit, the named plane or landmark. A maximum extent is not a diameter; a first slice that works is not a plane.
4. Each hypothesis with its bar, taken from a published value or a power calculation, never from the data.
5. What counts as failure, and the sentence you will write if it fails.
6. The analysis plan: held-out sets, who reads what blind.
7. A freeze block: date, author, "no further edits".

```
sha256sum PREREG.md > PREREG.md.sha256
git add PREREG.md PREREG.md.sha256 && git commit -m "PREREG v1 frozen"
# first line of every producing job:
sha256sum -c PREREG.md.sha256 || { echo "STOP: PREREG.md changed since the
freeze"; exit 90; }
```

Data rules that never bend

Rule	Statement	Why
IRB umbrella	A dataset is under HPR 2026-269 only if all five hold: public and de-identified by the source; downloadable without application, permission or data-use agreement; not restricted-use; no re-identification key; no linkage attempt. Anything else needs its own filing before use.	one dataset was used before the separate filing the umbrella promised
Licences	CTSpinoPelvic1K labels CC BY-NC-SA 4.0, CT CC BY 3.0, code Apache-2.0. Cite the DOI. Never redistribute volumes; `.gitignore` blocks `*.nii.gz`.	the SA clause is inherited and cannot be dropped

Rule	Statement	Why
Private forever	Level-identification code, FluoroCap, model weights, spinesurg-ct-nnunet, XRSpinoPelvic1K. Never in a public repository, never on the website.	the website shipped 121 MB of weights until 2026-09-14
Credentials	Read from files, never echoed, never in `sbatch --export`, never committed. Students use their own Hugging Face read token.	a token on a command line is in the history and the job log
Personal data	Survey CSVs, contact spreadsheets, staff lists: never in a repository.	a staff list with emails sat in a public folder for 171 commits
Labels	Read identifiers from the release's `dataset_labels.json`; call `labels_for(volume)`; never import a fixed map. The wrong map returns T4 for `femur_left` without raising. Id 29 (the v10 carved S1) is retired in v11; the public mirror is still v10 on 2026-09-27.	pelvic incidence was computed for the whole release against the legacy map
Volumes	Canonicalise to analyse; never reorient to write. Volumes are stored P,I,R.	`as_closest_canonical` + `nib.save` transposed a label away from its CT
Case ids	Zero-padded in the release; `zfill(4)` before any join.	every row looked changed
Two tracks	Students, the grid and non-commercial data are the research track; nothing moves from it to a commercial track.	written policy

Code and environment

An empty, missing, ambiguous or defaulted input stops the run. This one habit would have prevented 16 of the 29 incidents tallied on 2026-09-27: an empty held-out set that flattered every result, a wrong label map, a retired gate that swapped a figure panel, a

trimming step that exited 0 with empty output, a dictionary parsed as a list that reported zero annotators.

- Scripts are files in the repository: no heredocs over ssh, no `sbatch --wrap`, no `python -c` with backslashes. On Windows, network calls go through `wsl -e bash -lc`.
- `set -uo pipefail`, not `set -e`: with `-e` a job dies without saying which check failed. Write each check with its own message and exit code.
- Required variables `${LIST:?}`; every input exists and is non-empty; expected counts match; `sha256sum -c` on frozen inputs. In Python, a lookup that can return `None` raises.
- Pin the environment: containers by digest or a pre-unpacked sandbox path, never `:latest`. `make env-lock` exports the grid environment with commit, host, interpreter, torch and CUDA, and refuses a lock that came back without the torch line. Regenerate it at every freeze.
- Write results to a `.part` name and rename into place after your checks pass.

Every results file carries a `_run` block written by the producing script (`run_record()` in `RECIPE.md` §4). It refuses outside a git checkout and refuses uncommitted changes unless `ALLOW_DIRTY=1`.

`_run` field	Value
job, task	Slurm job id and array task id
host, utc	node name and UTC time
git_head, git_dirty	the commit, and whether the tree was clean
script, script_sha256, argv	the producing script, its hash, its arguments
inputs	path → sha256 of every input read
prereg_sha256	the frozen pre-registration hash

The grid

The rules apply to every job, including a ten-record test. Each one was paid for.

#	Rule	The incident
1	The login node runs only `sbatch`, `squeue`, `scancel`, `ls`, `tail`, `git`. Downloads are jobs too; compute nodes have internet.	dozens of smoke tests on a node shared by ~95 users (2026-09-21)
2	`sbatch -D <dir>`: `--output` resolves against the submit directory.	logs where nobody looked
3	`--dependency=afterany`, never `afterok`; the child reads a file to decide whether the parent succeeded.	an 8 h limit made a whole chain DependencyNeverSatisfied (2026-09-09)
4	`/tmp` is node-local and swept. Scratch and IPC under `/dev/shm/${USER}_<name>_${SLURM_JOB_ID}` with a `trap` to remove it.	sockets of two 18-hour runs deleted at the same minute
5	Large outputs on the lab's CephFS volume, not home. Home is a soft quota read with `pan_quota`.	home at 97 %; 260 GB deleted; ~90 scripts then read an empty held-out set
6	CPU `--partition=reqp --qos=requeue`; GPU `--partition=gmsap,gvohp,galhp --qos=gpu --gres=gpu:1`.	GPU partitions accept only the `gpu` QOS
7	Containers: the conda singularity, a pre-unpacked sandbox, the specific CephFS bind, `/tmp` bound to scratch; probe in step 0.	three jobs lost in a row: exit 127, starter not found, bad bind
8	Arrays share one list through atomic claims, stale takeover, heartbeat, a GPU self-test before the first case, a failure-streak stop.	Blackwell nodes under a CUDA 12.4 build would have failed every claimed case
9	A dependent job whose inputs are not ready resubmits itself and exits 0; it never waits on a node.	design
10	`--no-requeue` unless the script is re-entrant; then `--requeue --open-mode=append`.	a restart must not mix two attempts' files
11	One LaTeX compile at a time per directory (`flock`).	two compiles interleaved on `main.aux`; a stale PDF stayed on disk

#	Rule	The incident
12	Before cancelling anything, record every job with its script header as the resume instruction.	Greg, 2026-09-27
13	`sbatch --parsable`; ids with timestamps and replacement notes into `JOBS.txt`.	ids that lived only in a terminal
14	A pre-registered decision gate between a timing run and a production array.	cost control on a 21,622-CT census

```
#!/bin/bash
# <PROJECT>, <task>: what this job produces and where. Resumable:
resubmitting skips finished cases.
#   run: sbatch -D <workdir> --array=0-23 --export=ALL,LIST=<list>
slurm/<task>.sh
#SBATCH --partition=reqp
#SBATCH --qos=requeue
#SBATCH --no-requeue
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=04:00:00
#SBATCH --output=logs/%x_%A_%a.out
set -uo pipefail
LIST=${LIST:?set LIST}
W=/dev/shm/${USER}_<task>_${SLURM_JOB_ID:-x}_${SLURM_ARRAY_TASK_ID:-0}; mkdir
-p "$W"; trap 'rm -rf "$W"' EXIT
export PYTHONNOUSERSITE=1 TMPDIR=$W
sha256sum -c PREREG.md.sha256 || { echo "STOP: PREREG.md changed"; exit 90; }
for p in "$LIST" code/<task>.py; do [ -s "$p" ] || { echo "REFUSING: missing
or empty $p"; exit 1; }; done
python code/<task>.py --list "$LIST" --out results/ --shard
"${SLURM_ARRAY_TASK_ID:-0}" --nshards "${SLURM_ARRAY_TASK_COUNT:-1}"
```

Estimate before you submit (per-item cost × items ÷ parallelism + queue wait), write the ids and the ETA into `JOURNAL.md` and the checkpoint, then stop.

Nextflow

Use Nextflow for a pipeline of three or more stages over many cases, mostly on CPU, mostly other people's tools: the nf-core pipelines pinned by revision (rnaseq 3.22.2, differentialabundance 1.5.0, fetchngs 1.12.0, methylarray). A GPU census that shares one list across mixed GPU types is already built as Slurm arrays with atomic claims; do not rewrite it. None of the imaging repositories runs Nextflow today.

```
process {
  executor = 'slurm'; queue = 'reqp'; clusterOptions = '--qos=requeue'
  maxRetries = 3
  errorStrategy = { task.exitStatus in ((130..145) + 104 + (175..177)) ?
'retry' : 'finish' }
  resourceLimits = [ cpus: 40, memory: 240.GB, time: 96.h ]
}
executor { queueSize = 50; submitRateLimit = '20/1min' }
singularity { enabled = true; autoMounts = true; cacheDir =
'<CephFS>/singularity_cache'
              runOptions = '-B <CephFS>/<project>/tmp:/tmp'; pullTimeout =
'60 min' }
```

The head job is one `sbatch` on `reqp --qos=requeue --requeue --open-mode=append`, 2–4 CPUs, days of wall time. It owns one launch directory per run on CephFS, keeps `NXF_HOME`, temp and the container cache on CephFS, always passes `-resume`, writes the exit code to `NEXTFLOW_EXIT`, and refuses to start on a dirty tree. A check job chained `afterany` reads that file before anything is deleted.

Change	Re-runs the task?	What we do
the `script:` text after interpolation	yes	—
a `bin/` script named in the script block	yes	—
a `path` input (path, mtime, size)	yes	copy with `cp -a` to avoid spurious re-runs
a `val` input or an `ext.args` string	yes, by value	—

Change	Re-runs the task?	What we do
the content behind a path passed as a string	no	name the file by its own hash and verify name == content every run
the content behind a container tag or sandbox	no	pin by digest; record it in the result
files read by absolute path but not declared	no	frozen copies, hashed outside Nextflow
`cpus`, `memory`, `time`, `queue`, `publishDir`	no	—
outputs already in `storeDir`	skipped regardless	never `storeDir` for a stage whose script may change
a different launch directory	nothing resumes	one launch directory per run

Nextflow's task hash never reaches the paper. The producing task still writes the `_run` block into its result, and the trace is enabled with `hash,native_id,process,tag,status,exit,attempt,container,workdir,script`. A minimal segment → measure → manifest → compile skeleton is in `RECIPE.md` §6 (untested).

From results to manuscript

Nothing is typed by hand between the results file and the PDF. This is the chain the SchwingNet and U251 manuscripts compile through.

1. `results/<task>.json`: the numbers plus `_run`.
2. `numbers_manifest.json`: one entry per quoted number. `pointer` is an RFC 6901 JSON pointer; `job` is a Slurm job id, digits only; `prereg` names the hypothesis it answers.
3. In the tex, `\RESULT{key}` and nothing else, in tables as well as prose. Tables are written by a tool from the manifest. On 2026-09-27 the SchwingNet prose had 0 bare result numerals and the tables 167, because the check scanned only the prose.
4. The build runs the same steps every time, on the grid, one compile per directory.

5. Every paper has one reproduce target that submits the arms, collects, prints the tables the document quotes, compares them with a headline dictionary, and exits non-zero on MISSING or MOVED. "A reproduction that cannot fail is not a reproduction."

```
"naming_lat_whole_cv": {
  "file": "results/eval_gate/closure/cv_v9/cv_v9.json",
  "pointer": "/families/pre:cv5_det/cells/LAT/kfull/exact_pct",
  "fmt": "pct1", "kind": "number", "job": "40456123", "prereg": "H1",
  "note": "Table 3 row 1; case-grouped five-fold"
}
```

Build step	What it refuses
`results_macros.py`	writes `generated/numbers.tex` and `numbers_audit.md` (value, file, sha256, time per key); a missing result prints red `XXX Result Pending-<job>`
`results_macros.py --check`	any pending key, unused key, or bare numeral in any tex file
`compose_figures.py --check`	a figure panel whose sha256 differs from the recorded one
`check_limits.py`	the journal's word, abstract, figure, table and reference limits, measured on the built PDF
`check_claim_checklist.py`	a claim in `CLAIMS_LEDGER.md` without its evidence file and verdict
`latexmk` under `flock`	a second compile in the same directory

Two copies (double-anonymised review, camera-ready) come from one source through `\ifcameraready`. Page and word limits are what the build measures on the published format; the submission form's count is meaningless. Captions are at most 60 words; the argument belongs in the text.

Checking a number

The recurring failure is a number that is wrong but plausible. Three estimators produced finite, reasonable values from the wrong bone, the wrong surface or the wrong label map, and none raised. Internal consistency caught none of them: PI = SS + PT held through an

error that moved sacral slope by 9°, because both shared one bad landmark.

A number is checked against something outside the computation, and the check is committed beside the claim.

1. A published value. Every measure compares itself with a normative series; anything that fails is withheld, not disclosed as a caveat.
2. Spread, not the median. Canal depth had medians near the published means while 21 % of L3 canals read below any value reported in a living adult. Compare the SD with the widest published SD, and the share below a threshold with the published prevalence.
3. Reproduce the definition, not the units: a midsagittal chord at a named plane, not a maximum extent on a first slice that works.
4. Render the mask. Three analytically reasonable fixes to one measurement were all wrong; one render showed what was there.
5. Test the tidy explanation. The innocent reading of 28 % identity violations was wrong; the distribution showed a second surface.
6. An independent implementation where one exists.
7. The verifier is committed beside the claim. The scripts that re-read 375 targets, ordered a byline from the ledgers and generated the job record all lived in scratch folders; one had earlier reported "nobody is missing" from a dictionary parsed as a list.
8. Say plainly what is not known. "An unexplained failure in 128 records" is the preferred form.

Before a freeze, four audits: numbers (`results_macros.py --check`), figures (`compose_figures.py --check`), claims (`CLAIMS_LEDGER.md`), identity (`front_matter.json` , every field with `source` and `confirmed` ; an unconfirmed field holds the strict build at FAIL).

Working with Claude

A session ends when its limit is hit, so the state of the work lives in files, never in the conversation.

Session start. `git status` locally; `ssh -o BatchMode=yes grid hostname` (expect `warrior`); `squeue -u $USER` ; `git pull` on the grid; read the top block of `RESUME.md` and the last five lines of `JOURNAL.md` ; run `pipeline_state.py` where the project has one. Before any step, check whether its output already exists.

Submit, estimate, checkpoint, stop. An agent that submits returns the job ids, an ETA and the files to collect; that goes into the checkpoint and the turn ends. No `squeue` loop beyond about ten minutes: on 2026-09-24 polling agents burned a session limit four times in one day while the grid did the work. On "reactivate", one collect pass reads the table, collects what finished, submits what is missing, updates the table, stops.

```
## CHECKPOINT 2026-09-27 17:40 EDT (grid clock): <one-line state> (Greg: "
<the directive, verbatim>")
DONE: <what exists: paths, job ids, timestamps, sha256 of every frozen input>
PENDING JOBS | job | what | submitted | ETA | output | collect step |
WHEN THE JOBS FINISH: 1. read <file> first; 2. <gate> stays pending until
<file> exists; 3. rerun <command> if <condition>
WORKFLOWS: | run id | what | where results land | + the resume call and the
journal path
DO NOT REDO: <what a fresh session must not repeat>
NOTHING COMMITTED BY ANY WORKFLOW (or the commit ids)
FOR GREG / WAITING ON: <decisions only the PI can make>
LIMITS TO STATE: <the confounds every write-up must carry>
```

Status is computed, not written. A hand-written PENDING table listed a job as pending three hours after `sacct` showed it cancelled. `tools/pipeline_state.py` declares for every stage the artefact whose existence means it finished and prints done / running / ready / blocked / failed from disk and queue. When you add a stage, add it there.

Memory is one fact per file with a Why (the incident) and a How to apply (the command), indexed in `MEMORY.md`; the first entry is the handoff note. Before cancelling anything, write the job record.

Leaving, handover and credit

Leaving a project is one pull request, merged by the lead within a week, that does three things.

1. Appends a `HANDOVER <date> <name>` block to `JOURNAL.md`: what is done (paths, job ids, hashes), what is pending, what must not be redone, where the frozen inputs are, and the one thing the next person will get wrong if nobody tells them.
2. Moves the name on the board card from members to alumni and names who takes each open task.
3. If the lead leaves, the co-lead becomes lead and a new co-lead is named in the same pull request. A project is never left without both.

Credit is recorded as the work proceeds: annotation ledgers, git history, `JOURNAL.md`, the board's task lines. Authorship follows the ICMJE criteria; among those who qualify, the middle of the byline is ordered by recorded volume, and first and senior authorship are set at the pre-registration and changed only in `DEVIATIONS.md`. Every author's fields live in `front_matter.json` with `source` and `confirmed`; nothing unconfirmed prints.

Checklists, and what is not yet true

Before the first number

- [ ] `PREREG.md` frozen and its hash committed; bars taken from published values
- [ ] the dataset passes the five IRB criteria or has its own filing; licence and DOI recorded
- [ ] identifiers read from the release's `dataset_labels.json`; exclusions listed with reasons
- [ ] `env/<name>.lock.txt` written from the grid; the board card exists with lead, co-lead, contact

Before a freeze

- [ ] every results file carries `_run`; every quoted number has a manifest key with a digit-only job and a `prereg` item
- [ ] `results_macros.py --check` passes on every tex file, tables included; every figure panel's hash recorded
- [ ] every mask behind a headline number rendered and looked at; each headline checked outside the computation, the check committed
- [ ] environment lock regenerated; code committed; the grid clone at the same commit

Before a submission

- [ ] `check_limits.py` passes on the built PDF; captions ⩽ 60 words; `CLAIMS_LEDGER.md` complete; `front_matter.json` all confirmed
- [ ] the reproduce target ran clean from the committed tree; `DEVIATIONS.md` current; failed bars reported as failures
- [ ] nothing private in the submission

What is not yet true (2026-09-27)

Gap	State	Closes when
tables through macros	SchwingNet tables carry 167 bare numerals; `--check` scanned only the prose	the check scans every tex file and the tables are tool-written
`job` is a job id	0 of 79 manifest keys begin with a Slurm job id	the loader refuses free text; entries repointed as tasks re-run
`_run` in every result	the manifest-path JSONs carry no commit, job id or script hash	producing tools call `run_record()` and re-run
`prereg` field	no manifest entry names its pre-registered item	added with the loader change
verifiers committed	`verify_numbers.py`, `_roster5.py`, the job-record generator were scratch scripts	committed beside their claims
a grid data path students can read	the CephFS group has two members; the PI's home is not group-readable	a group-readable export or ACL (Greg)
one authorship rule	three statements existed	the rule above; Greg confirms
v11 on the public mirror	Hugging Face still serves v10 (`dataset_labels.json` carries 29)	Greg publishes v11; student materials move to v11 identifiers
pelvis decision gate	written and pre-stated; never yet produced `DECISION.json`	the timing job completes
Nextflow for imaging	none of the imaging repositories runs it; the public tutorial says otherwise	the tutorial is corrected

Fixed with this recipe: the student templates (now `reqp --qos=requeue`, `/dev/shm` scratch, `set -u` with explicit checks, `afterany`), the project board, and the compile lock on the grid.